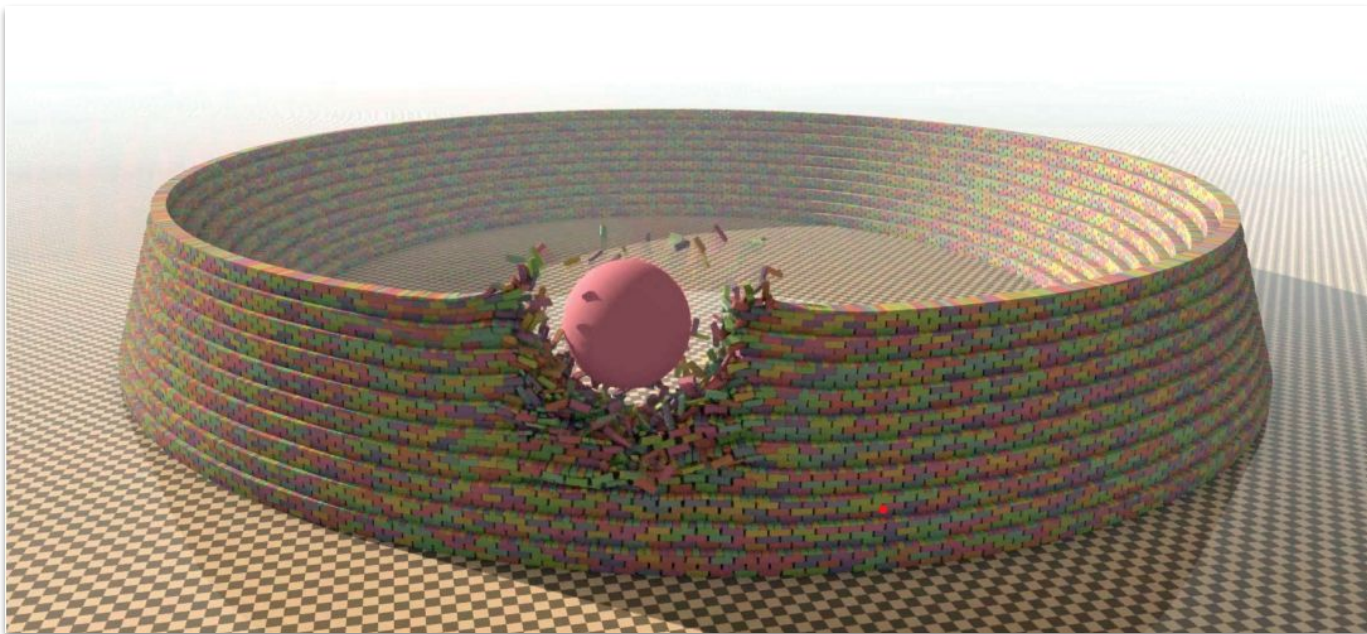


Augmented Vertex Block Descent

Chris Giles, Elie Diaz, Cem Yuksel
[2025]



LIRIS - Siggraph at Liris [08/07/2025]

[2014] Optimization Integrator for Large Time Steps

Optimization Integrator for Large Time Steps

Theodore F. Gast, Craig Schroeder, Alexey Stomakhin, Chenfanfu Jiang and Joseph M. Teran

Abstract—Practical time steps in today's state-of-the-art simulators typically rely on Newton's method to solve large systems of nonlinear equations. In practice, this works well for small time steps but is unreliable at large time steps at or near the frame rate, particularly for difficult or stiff simulations. We show that recasting backward Euler as a minimization problem allows Newton's method to be stabilized by standard optimization techniques with some novel improvements of our own. The resulting solver is capable of solving even the toughest simulations at the 24 Hz frame rate and beyond. We show how simple collisions can be incorporated directly into the solver through constrained minimization without sacrificing efficiency. We also present novel penalty collision formulations for self collisions and collisions against scripted bodies designed for the unique demands of this solver. Finally, we show that these techniques improve the behavior of Material Point Method (MPM) simulations by recasting it as an optimization problem.

Index Terms—Computer Graphics, Three-Dimensional Graphics and Realism, Animation

1 INTRODUCTION

THE most commonly used time integration schemes in use today for graphics applications are implicit methods. Among these, backward Euler [1], [2], [3], [4], [5] or variants on Newmark methods [6], [7], [8] are the most common, though even more sophisticated schemes like BDF-2 [9], [10], implicit-explicit schemes [11], [12], or even the more exotic exponential integrators [13] have received consideration. Integrators have been the subject of comparison before (see for example [3], [9], [14]), seeking good compromises between speed, accuracy, robustness, and dynamic behavior.

These integrators require the solution to one or more nonlinear systems of equations each time step. These systems are typically solved by some variation on Newton's method. Even the most stable simulators are typically run several time steps per 24 Hz frame of simulation. There is growing interest in running simulations at larger time steps [15], so that the selection of Δt can be made based on other factors, such as damping or runtime, and not only on whether the simulator works at all. One of the major factors that limits time step sizes is the inability of Newton's method to converge reliably at large time steps (See Figures 3, 2, and 4), or if a fixed number of Newton iterations are taken, the stability of the resulting simulation. We address this by formulating our nonlinear system of equations as a minimization problem, which we demonstrate can be solved more robustly. The idea that dynamics, energy, and minimization are related has been known since antiquity and is commonly leveraged in variational integrators [6], [12], [16], [17], [18], [19], [20]. The idea that the nonlinear system that occurs from methods like backward Euler can be formulated as a minimization problem has appeared many times in graphics in various forms [2], [4], [5], [13], [19], [19] point out that minimization leads to a method that is both simpler and faster than the equivalent nonlinear root-finding problem, and [5] show that a minimization formulation can be used to solve mass-spring systems more efficiently. [17] use a minimization formulation as a means of ensuring that a solution to their nonlinear system can be found assuming one exists. [21] shows that a minimization formulation can be used to enforce

constraints robustly and efficiently. [2] shows that supplementing Newton's method with a line search greatly improves robustness. [4] also shows that supplementing Newton's method with a line search and a definiteness correction leads to a robust solution procedure. Following their example, we show that recasting the solution of the nonlinear systems that result from implicit time integration schemes as a nonlinear optimization problem results in substantial robustness improvements. We also show that additional improvements can be realized by incorporating additional techniques like Wolfe condition line searches which curve around collision bodies, conjugate gradient with early termination on indefiniteness, and choosing conjugate gradient tolerances based on the current degree of convergence.

This publication is an extended version of [22] in which we have applied the optimization integrator approach to the MPM snow simulator of [23]. This allows us to take much larger time steps than the original method and results in a significant speedup.

2 TIME INTEGRATION

The equations of motion for simulating solids are

$$\dot{\mathbf{x}} = \mathbf{v} \quad \mathbf{M}\dot{\mathbf{v}} = \mathbf{f} \quad \mathbf{f} = \mathbf{f}(\mathbf{x}, \mathbf{v}),$$

where $\mathbf{f}$ are forces. As is common in graphics we assume $\mathbf{M}$ is a diagonal lumped-mass matrix. Since we are interested in robustness and large time steps, we follow a backward Euler discretization. This leads to

$$\frac{\mathbf{x}^{n+1} - \mathbf{x}^n}{\Delta t} = \mathbf{v}^{n+1} \quad \mathbf{M} \frac{\mathbf{v}^{n+1} - \mathbf{v}^n}{\Delta t} = \mathbf{f}^{n+1} = \mathbf{f}(\mathbf{x}^{n+1}, \mathbf{v}^{n+1}).$$

Eliminating $\mathbf{v}^{n+1}$ yields

$$\mathbf{M} \frac{\mathbf{x}^{n+1} - \mathbf{x}^n - \Delta t \mathbf{v}^n}{\Delta t^2} = \mathbf{f} \left(\mathbf{x}^{n+1}, \frac{\mathbf{x}^{n+1} - \mathbf{x}^n}{\Delta t} \right),$$

which is a nonlinear system of equations in the unknown positions $\mathbf{x}^{n+1}$. This system of nonlinear equations is normally solved with Newton's method. If we define

$$\mathbf{h}(\mathbf{x}^{n+1}) = \mathbf{M} \frac{\mathbf{x}^{n+1} - \mathbf{x}^n - \Delta t \mathbf{v}^n}{\Delta t^2} - \mathbf{f} \left(\mathbf{x}^{n+1}, \frac{\mathbf{x}^{n+1} - \mathbf{x}^n}{\Delta t} \right), \quad (1)$$

then our nonlinear problem is one of finding a solution to $\mathbf{h}(\mathbf{x}) = \mathbf{0}$. To do this, one would start with an initial guess $\mathbf{x}^{(0)}$, such as the

Implicit integration as optimization problem

$$M\ddot{x} = f(x) \quad \ddot{x} = \frac{\dot{x}^{n+1} - \dot{x}^n}{\Delta t} \quad \dot{x}^{n+1} = \frac{x^{n+1} - x^n}{\Delta t}$$

$$x^{n+1} \leftarrow \underset{x}{\operatorname{argmin}} G(x) \quad x^{n+1} = x^n - \frac{\partial^2 G(x)^{-1}}{\partial x^2} \frac{\partial G(x)}{\partial x}$$

Robustness

Versatility

Costly

Complex

• T. Gast, C. Schroeder, C. Jiang and J. Teran are with the University of California Los Angeles

• A. Stomakhin and J. Teran are with Walt Disney Animation Studios

Manuscript received November 12, 2014.

[2024] Vertex Block Descent

Vertex Block Descent

ANKA HE CHEN, University of Utah, USA
 ZIHEG LIU, University of Utah, USA
 YIN YANG, University of Utah, USA
 CEM YUKSEL, University of Utah & Roblox, USA



Fig. 1. Example simulation results using our solver, both of those methods involve more than 100 million DoFs and 1 million active collisions.

We introduce vertex block descent, a block coordinate descent solution for the variational form of implicit Euler through vertex-level Gauss-Seidel iterations. It operates with local vertex position updates that achieve reductions in global variational energy with maximized parallelism. This forms a physics solver that can achieve numerical convergence with unconditional stability and exceptional computation performance. It can also fit in a given computation budget by simply limiting the iteration count while maintaining its stability and superior convergence rate.

We present and evaluate our method in the context of elastic body dynamics, providing details of all essential components and showing that it outperforms alternative techniques. In addition, we discuss and show examples of how our method can be used for other simulation systems, including particle-based simulations and rigid bodies.

CCS Concepts: • **Computing methodologies** → **Physical simulation**; **Collision detection**.

Additional Key Words and Phrases: physics-based simulation, elastic body, rigid body, time integration

1 INTRODUCTION

Physics-based simulation is the cornerstone of most graphics applications and the demands from simulation systems to deliver improved stability, accelerated computational performance, and enhanced visual realism are ever-growing. Particularly in real-time graphics applications, the stability and performance requirements are so strict that realism can sometimes be begrudgingly considered of secondary importance.

Notwithstanding the substantial amount of research and ground-breaking discoveries made on physics solvers over the past decades,

existing methods still leave some things to be desired. They either deliver high-quality results, but fail to meet the computational demands of many applications or fit in a given computation time by sacrificing quality. Stability, on the other hand, is always a challenge, particularly with strict computation budgets.

In this paper, we introduce *vertex block descent (VBD)*, a physics solver that offers unconditional stability, superior computational performance than prior methods, and the ability to achieve numerical convergence to an implicit Euler integration. Though our method is a general solution that can be used for a variety of simulation problems, we present and evaluate it in the context of elastic body dynamics. Then, we briefly discuss how our method can be applied to some other simulation systems, including particle-based simulations and rigid bodies.

Our VBD method is based on block coordinate descent that performs vertex-based Gauss-Seidel iterations to solve the variational form of implicit Euler. For elastic body dynamics, each iteration runs a loop over the mesh vertices, adjusting the position of a single vertex at a time, temporarily fixing all others. This offers maximized parallelism when coupled with vertex-based mesh coloring, which can achieve an order of magnitude fewer colors (i.e. serialized workloads) as compared to element-based parallelization. Our local position-based updates can ensure that the variational energy is always reduced. Therefore, our method maintains its stability even with a single iteration per time step and large time steps, operating with unconverged solutions containing a large amount of residual. With more iterations, it converges faster than its alternatives. Thus, it can more easily fit in a given computation budget, while maintaining stability with improved convergence.

We present all essential components of using VBD for elastic body dynamics, including formulations for damping, constraints,

Local optimization (per vertex)

$$G_i(\mathbf{x}) = \frac{m_i}{2h^2} \|\mathbf{x}_i - \mathbf{y}_i\|^2 + \sum_{j \in \mathcal{F}_i} E_j(\mathbf{x})$$

$$\mathbf{x}_i \leftarrow \underset{\mathbf{x}_i}{\operatorname{argmin}} G_i(\mathbf{x}) \quad \Delta \mathbf{x}_i = \underbrace{\left[\frac{\partial^2 G_i(\mathbf{x})}{\partial \mathbf{x}_i^2} \right]^{-1}}_{\substack{3 \times 3 \\ \text{Hessian Matrix}}} \underbrace{\frac{\partial G_i(\mathbf{x})}{\partial \mathbf{x}_i}}_{\substack{3 \times 1 \\ \text{Force Vector}}}$$

Robust

Versatile

Simple

Efficient

Can't do hard constraint

Hard constraints ?

We want to some particles to satisfy a condition

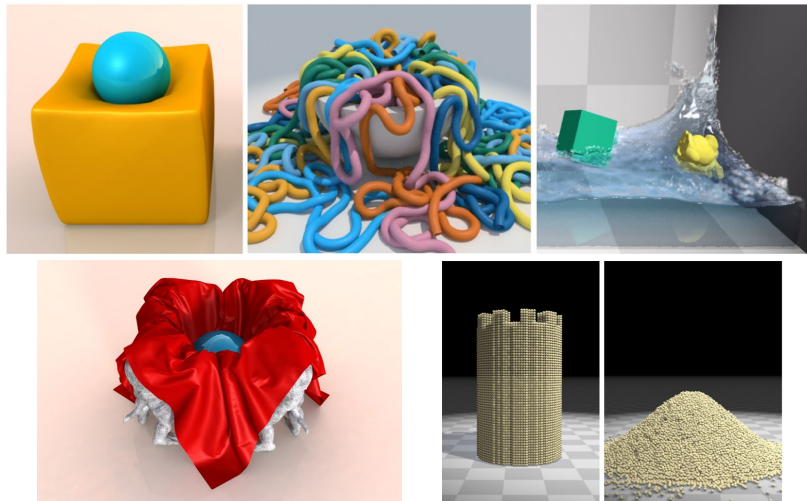
bilateral

$$C(x) = 0$$

unilateral

$$C(x) \geq 0$$

Can be pretty much anything

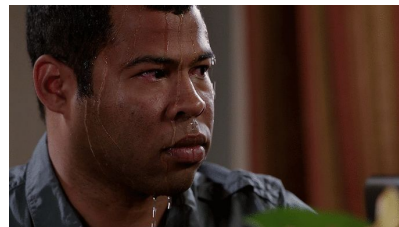


Hard constraints are needed for collisions

But they have infinite stiffness !

Constraints in VBD

$$E_j(\mathbf{x}) = \frac{1}{2}k_j \left(C_j(\mathbf{x}) \right)^2$$



This formulation can't represent hard constraint

Augmented Vertex Block Descent

Augmented Vertex Block Descent

CHRIS GILES, Roblox, USA
ELIE DIAZ, University of Utah, USA
CEM YUKSEL, University of Utah, USA

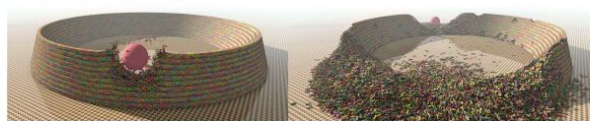


Fig. 1. Our augmented vertex block descent method allows simulating hard constraints, which are critical for handling contacts and stacking, shown here simulating a pile of 110,000 blocks smashed by a sphere using 4 iterations, taking 3.5 ms (9.8 ms including collision detection) per frame on an NVIDIA RTX 4090 GPU.

Vertex Block Descent is a fast physics-based simulation method that is unconditionally stable, highly parallelizable, and capable of converging to the implicit Euler solution. We extend it using an augmented Lagrangian formulation to address some of its fundamental limitations. First, we introduce a mechanism to handle hard constraints with infinite stiffness without introducing numerical instabilities. Second, we substantially improve the convergence in the presence of high stiffness ratios. These changes we introduce allow simulating complex contact scenarios involving rigid bodies with stacking and friction, articulated bodies connected with hard constraints, including joints with limited degrees of freedom, and stiff systems interacting with soft bodies. We present evaluations using a parallel GPU implementation that can deliver real-time performance and stable simulations with low iteration counts for millions of objects interacting via collisions, various joint attachment constraints, and springs of various stiffness. Our results show superior performance, convergence, and stability compared to the state-of-the-art alternatives.

CCS Concepts: • Computing methodologies → Physical simulation.

Additional Key Words and Phrases: Physics-based animation, simulation

ACM Reference Format:

Chris Giles, Elie Diaz, and Cem Yuksel. 2025. Augmented Vertex Block Descent. *ACM Trans. Graph.* 44, 4, Article 112.2213 (August 2025), 12 pages. <https://doi.org/10.1145/3731195>

1 INTRODUCTION

Physics-based simulation is the cornerstone of many graphics applications and various simulation methods have been developed over the years for different systems/problems. Recently, the *vertex block descent* (VBD) [Chen et al. 2024a] method presented superior

Authors' addresses: Chris Giles, giles@roblox.com, Roblox, San Mateo, CA, USA; Elie Diaz, elie.diaz@utah.edu, University of Utah, Salt Lake City, UT, USA; Cem Yuksel, cem@cs.utah.edu, University of Utah, Salt Lake City, UT, USA.

Please use nonam option or ACM Engage class to enable CC-BY license.
This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2025 Copyright held by the owner(s).
ACM 978-0-001-2023-9 ART112.2213
<https://doi.org/10.1145/3731195>

convergence behavior over prior techniques for soft body dynamics, delivering higher performance, improved parallelism, unconditional stability, and the ability to converge to the implicit Euler solution with the desired accuracy. In addition, VBD was shown to handle other simulation problems, such as particle systems and rigid body dynamics. Because VBD is fast and remains stable with few iterations, it is particularly useful for real-time graphics applications with strict computation budgets. Since VBD is a *primal* method [Macklin et al. 2020], it can also easily handle high mass ratios, which is another important advantage.

However, VBD is not without shortcomings. First of all, it struggles to converge for problems involving high stiffness ratios. Furthermore, there is no mechanism for enforcing explicit hard constraints (unless the corresponding degree of freedom can be entirely eliminated). Unfortunately, such cases are not uncommon. Collisions, for example, often require highly stiff penalty forces or hard constraints. It is also common to connect objects with hard constraints to form articulated rigid bodies or other mechanical systems. Thus, these issues prevent widespread use of VBD.

In this paper, we directly address these limitations of VBD and extend it with an *augmented Lagrangian* formulation that allows efficiently handling hard constraints via progressively increasing stiffness, without introducing numerical instability into the optimization through extreme stiffness values. This adds a *dual* step after each *primal* iteration of VBD, making our approach a hybrid, *primal-dual* method. To substantially improve convergence rate in the presence of high stiffness ratios, we apply the same progressive stiffness increment used in augmented Lagrangian to arbitrary, finite stiffness forces. Due to the use of augmented Lagrangian techniques, we call our method *Augmented-VBD* or *AVBD* for short.

Our evaluation involves challenging experiments with hard constraints, such as articulated rigid bodies and attachment constraints for soft bodies, extreme stiffness ratios, and complex collisions and contacts, such as rigid body stacking, piling, and friction. Our results show that AVBD can successfully handle these complex interactions, requiring a small number of iterations to maintain the constraints

Add perturbed / augmented Lagrangian

$$E_j^{(n)}(\mathbf{x}) = \underbrace{\frac{1}{2} k_j^{(n)} \left(C_j(\mathbf{x}) \right)^2}_{\text{"soft"}} + \underbrace{\lambda_j^{(n)} C_j(\mathbf{x})}_{\text{"hard"}}$$

Soft and hard approach in one pipeline

But we still have infinite stiffness ? So ...



Primal/Dual variables

Start

$$k_j^{(0)} = k_{\text{start}} \quad \lambda_j^{(0)} = 0$$

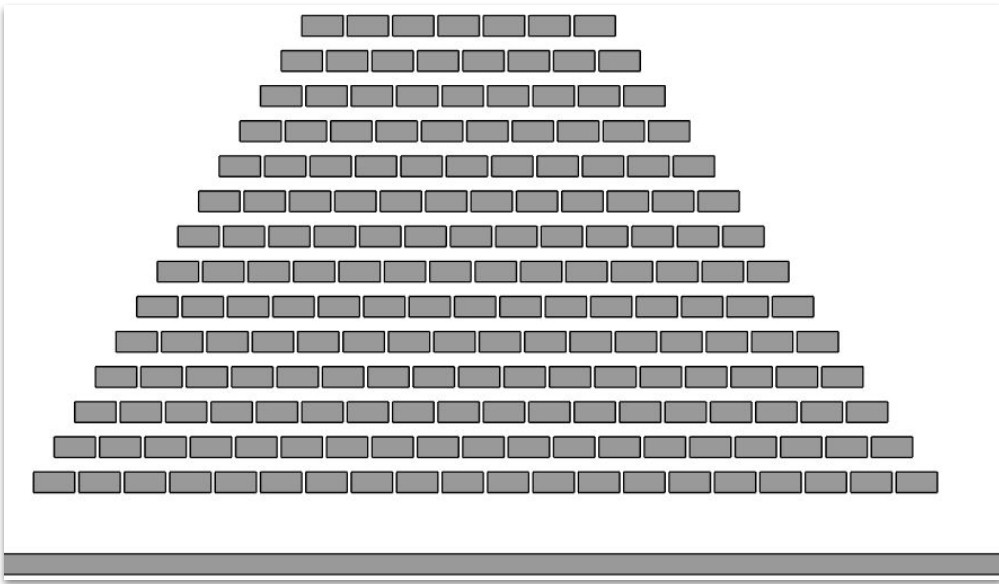
Update

$$k_j^{(n+1)} = k_j^{(n)} + \beta |C_j(\mathbf{x})|$$

$$\lambda_j^{(n+1)} = k_j^{(n)} C_j(\mathbf{x}) + \lambda_j^{(n)}$$

$$\beta \in [0, \infty[$$

Energy / Force increase gradually



Kaboom !
Stable but too much energy

Fine tuning

Warm Start

$$k_j^{(0)} = \max \left(\gamma k_j^t, k_{\text{start}} \right) \quad \lambda_j^{(0)} = \alpha \gamma \lambda_j^t$$

Use previous solution to get a better start

$$\gamma \in [0, 1]$$

Better convergence

Finite stiffness correction (soft)

$$k_j^{(n+1)} = \min \left(k_j^*, k_j^{(n)} + \beta |C_j(\mathbf{x})| \right)$$

Increase stiffness for soft constraint too and clamp by target value

Avoid stiffness ratio convergence problem

Prevent explosive error correction

$$C_j(\mathbf{x}) = C_j^*(\mathbf{x}) - \alpha C_j^*(\mathbf{x}^t)$$

Limit position correction compared to previous iteration

Avoid momentum injection

Correction on multiple frame

Implementation Details

Constraint's Hessian

$$\mathbf{H}_{ij} = k_j^{(n)} \left(\frac{\partial C_j(\mathbf{x})}{\partial \mathbf{x}_i} \right)^T \frac{\partial C_j(\mathbf{x})}{\partial \mathbf{x}_i} + \mathbf{G}_{ij}$$

$$G_{ij} = \lambda_j^+ \frac{\partial^2 C_j(x)}{\partial x^2}$$

$$\lambda_j^+ = k_j^{(n)} C_j(\mathbf{x}) + \lambda_j^{(n)}$$

Hessian must be positive definite

Easily not the case for hard constraint

Bad convergence in this case

Constraint's Hessian Approx

$$H_{ij} = k_j^{(n)} \left[\frac{\partial C_j(x)}{\partial x_i} \right]^T \frac{\partial C_j(x)}{\partial x_i} + \tilde{G}_{ij}$$

$$\tilde{\mathbf{G}}_{ij} = \text{diag}(\mathbf{g}_{ij}) \text{ norm of columns}$$

[2017] Geometric Stiffness for Real-time Constrained Multibody Dynamics

Quasi-Newton Approach

Only good for first iterations

Good enough for VBD !

Summary

$$E_j^{(n)}(\mathbf{x}) = \frac{1}{2} k_j^{(n)} \left(C_j(\mathbf{x}) \right)^2 + \lambda_j^{(n)} C_j(\mathbf{x})$$

$$k_j^{(0)} = \max \left(\gamma k_j^t, k_{\text{start}} \right)$$

$$k_j^{(n+1)} = \min \left(k_j^*, k_j^{(n)} + \beta |C_j(\mathbf{x})| \right)$$

$$C_j(\mathbf{x}) = C_j^*(\mathbf{x}) - \alpha C_j^*(\mathbf{x}^t)$$

$$H_{ij} = k_j^{(n)} \left[\frac{\partial C_j(x)}{\partial x_i} \right]^T \frac{\partial C_j(x)}{\partial x_i} + \tilde{G}_{ij}$$

Augmented Lagrangian

handle hard constraint

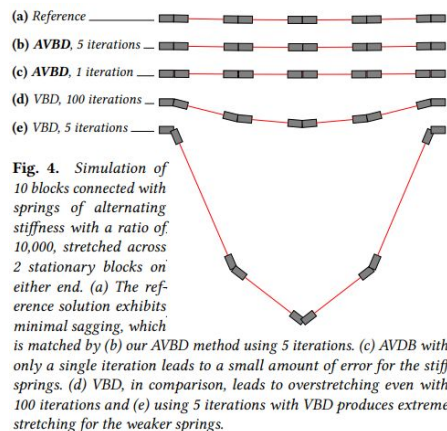
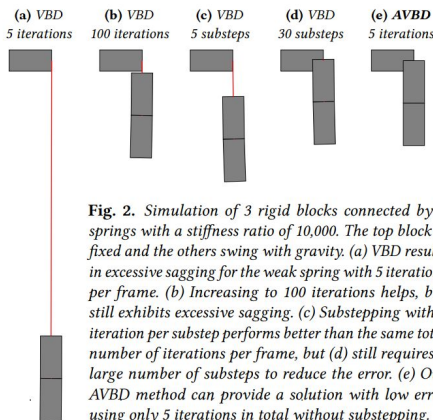
Minor Improvement

better convergence
avoid stiffness ratio problem
reduce energy gain

Hessian Approximation

Guaranty positive definite hessian

Honorable mention



	Range	Used	Description
β	$(0, \infty)$	10	Stiffness ramping controlling convergence rate
α	$[0, 1]$	0.95	Regularization to prevent explosive error correction
γ	$[0, 1)$	0.99	Warm starting of stiffness and dual variables



Video

Demo

Paper